

# Trabajo Práctico 2: Deep Q-Network

Lucio Luque Materazzi

lluquematerazzi@udesa.edu.ar

Ingeniería en Inteligencia Artificial

Aprendizaje reforzado

Otoño 2026

## 1. Resultados

En este trabajo se presentan los resultados del entrenamiento y evaluación de distintos algoritmos como Q-Learning, DQN y Reinforce para distintos ambientes *custom* como también de FrozenLake y CartPole.

Para los ambientes custom no se realizaron búsquedas de hiperparámetros, ya que su función fue evaluar si los algoritmos estaban implementados correctamente.

A su vez, se presentan los resultados de un agente con distintas mejoras realizadas sobre DQN para el ambiente MountainCar.

### 1.1. Ambientes custom

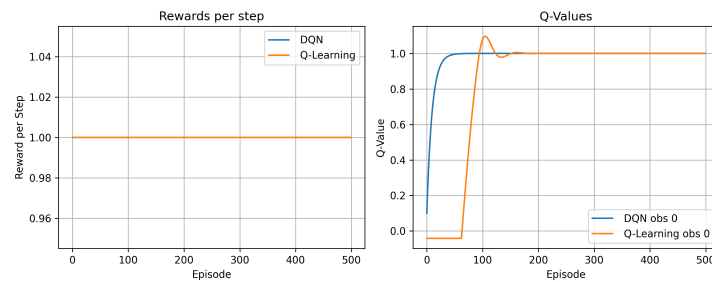


Figura 1: Recompensa por pasos totales y *Q-Values* por episodio para DQN y Q-Learning del ambiente *ConstantReward*.

En la Figura 1 se puede observar que la recompensa es siempre 1, lo que indica que el ambiente fue modelado correctamente. En cuanto a los *Q-Values*, ambos modelos convergen al valor esperado de 1, lo que confirma que el cálculo de la pérdida y la configuración del optimizador son correctos en ambos casos. Un detalle a destacar es que la convergencia del caso tabular resulta más suave que la del caso con redes neuronales: mientras que la tabla se corrige directamente mediante indexación, la red neuronal presenta mayor dificultad para converger debido a la cantidad de parámetros y conexiones involucradas.

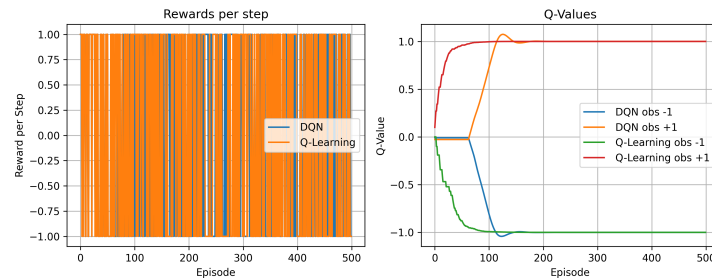


Figura 2: Recompensa por pasos totales y *Q-Values* por episodio para DQN y Q-Learning del ambiente *RandomObsBinaryReward*.

En la figura 2 se puede observar que la recompensa varía entre -1 y +1, lo cual es esperable dado que el ambiente asigna recompensas aleatorias dentro de ese rango. En cuanto a los *Q-Values*, cada uno

converge al valor esperado según la observación correspondiente, lo que sugiere que el flujo de gradientes y la retropropagación dentro de la red neuronal no presentan errores.

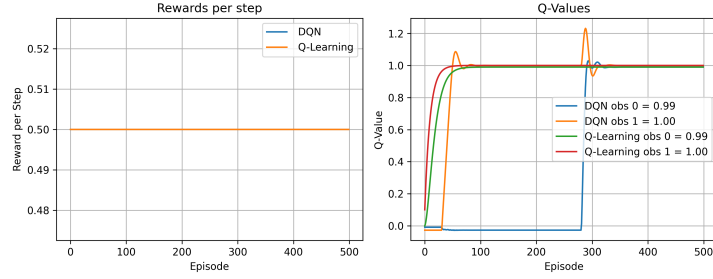


Figura 3: Recompensa por pasos totales y  $Q$ -Values por episodio para DQN y Q-Learning del ambiente *TwoStepDelayedReward*.

En la figura 3 se puede observar que la recompensa promedio por paso es 0.5 al ser un ambiente de dos pasos y recompensa de valor 1 al finalizar. Los  $Q$ -Values convergen al valor esperado según la observación, lo que confirma que el mecanismo de descuento y la acumulación de recompensas a lo largo del episodio fueron implementados correctamente en ambos algoritmos. La diferencia de velocidad de convergencia entre el caso tabular y el caso con redes neuronales es más notoria para este caso que en los ambientes anteriores: el  $Q$ -Value de la observación 0 para DQN tarda aproximadamente 300 episodios en converger. Esto es esperable dado que esa observación requiere esperar al final del episodio para recibir la señal y retropropagarla. Sin embargo, con una mejor búsqueda de hiperparámetros, la tardanza de la convergencia probablemente se reduciría.

## 1.2. Q-Learning para FrozenLake

|   | $\alpha$    | $\varepsilon$ | Success Rate |
|---|-------------|---------------|--------------|
| 1 | <b>0.01</b> | <b>0.4</b>    | <b>0.76</b>  |
| 2 | 0.05        | 1.0           | 0.76         |
| 3 | 0.1         | 1.0           | 0.76         |
| 4 | 0.5         | 0.4           | 0.74         |
| 5 | 0.005       | 0.4           | 0.37         |
| 6 | 0.001       | 0.4           | 0.25         |
| 7 | 0.0005      | 0.4           | 0.25         |
| 8 | 0.0001      | 0.4           | 0.25         |

Cuadro 1: Success rate para distintas configuraciones de  $\alpha$  y  $\varepsilon$  sobre 100 episodios de evaluación.

Para la búsqueda de hiperparámetros se evaluaron 8 valores de  $\alpha$  combinados con 11 valores de  $\varepsilon \in \{0.0, 0.1, \dots, 1.0\}$ , formando 88 combinaciones. Para cada  $\alpha$  se seleccionó el  $\varepsilon$  que maximizó el *success rate* en 100 episodios de evaluación. Los resultados se presentan en la tabla 1. Se observa que valores de  $\alpha$  muy pequeños ( $\leq 0.005$ ) resultan en tasas de éxito bajas, por lo que no logran converger a la  $Q$  óptima. De los mejores resultados se selecciona  $\alpha = 0.01$  y  $\varepsilon = 0.4$  como configuración final, por lograr el mejor rendimiento con una exploración moderada. Resulta llamativo que combinaciones que presentan  $\varepsilon = 1.0$  alcancen idéntico success rate, pudiendo deberse a la estocasticidad del ambiente.

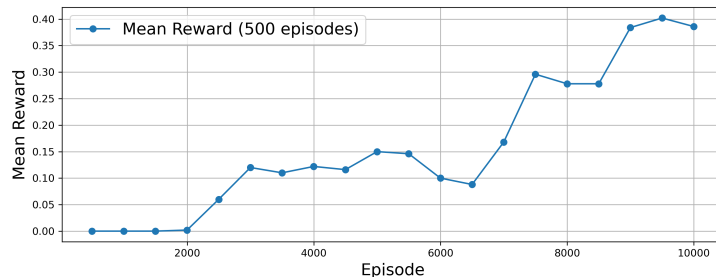


Figura 4: Recompensa promedio cada 500 episodios a lo largo del entrenamiento.

Luego se implementó  $\varepsilon$ -decay fijando los demás hiperparámetros y tomando, por la tabla 1,  $\alpha = 0.01$ . Al realizar una búsqueda, el mejor modelo resultó ser con  $\min \varepsilon = 0.1$ ,  $\max \varepsilon = 0.9$ , decay rate = 0.2. En la figura 4 se presenta la recompensa promedio cada 500 episodios para esta configuración durante el entrenamiento. Esta curva muestra una tendencia creciente a lo largo del entrenamiento. Sin embargo, llama la atención que el pico se alcance en torno a 0.4 y que en los últimos 500 episodios se produzca una leve caída, lo que podría indicar cierta inestabilidad al final del entrenamiento.

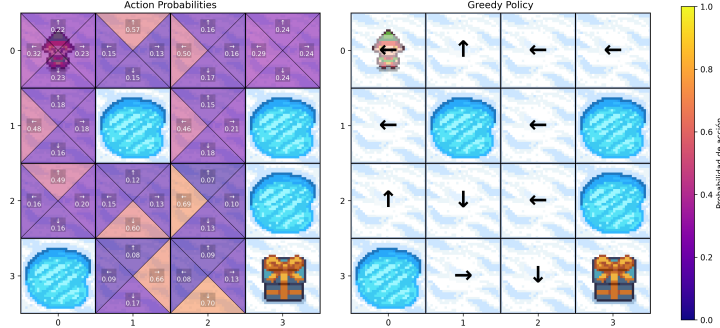


Figura 5: Q-Values finales para cada acción desde cada celda y política determinística final.

En la figura 5 se puede observar la tabla Q convergida. Dado que el deslizamiento está activado, el agente aprende a moverse en la dirección que minimice el riesgo de caer en un lago: con deslizamiento, hay 1/3 de probabilidad de ir en la dirección elegida y 1/3 para cada uno de los costados. Por eso, por ejemplo, en la primera columna, segunda fila, la acción preferida es ir hacia la pared en lugar de hacia abajo, reduciendo así el riesgo. La única celda que no converge a la acción óptima es la columna 3 fila 0, que debería apuntar hacia arriba y no hacia la izquierda. Esto se explica probablemente por ser una celda poco visitada durante el entrenamiento. Con hiperparámetros que favorezcan mayor exploración, es posible que el agente aprenda la acción correcta para esta celda.

### 1.3. DQN - CartPole

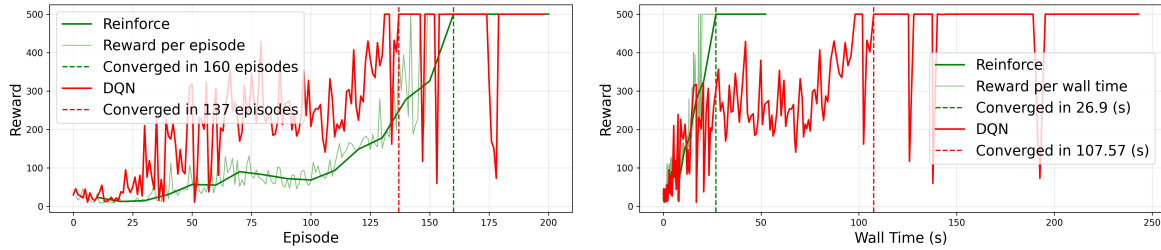


Figura 6: DQN vs REINFORCE en reward por episodio y wall time.

En la figura 6 se puede observar que DQN alcanza por primera vez un reward de 500 en el episodio 137 y que las oscilaciones posteriores corresponden probablemente a su política epsilon-greedy, que REINFORCE no presenta, y a la inestabilidad durante el entrenamiento. Considerando la cantidad de episodios (obviando las oscilaciones posteriores), DQN converge antes. Sin embargo, en términos de tiempo, REINFORCE lo hace aproximadamente 4 veces más rápido. En comparación, REINFORCE resultó ser más simple, liviano y sin necesidad de utilizar un replay buffer consumiendo así menos memoria. Por otro lado, DQN resultó ser considerablemente más sensible a los hiperparámetros y cuenta con más de ellos que REINFORCE.

Por último, a pesar de la aparente inestabilidad de DQN durante el entrenamiento, al evaluarlo sobre 100 episodios, alcanzó un 100% de éxito.

### 1.4. DQN mejoras - MountainCar

Se incorporaron ocho mejoras al agente DQN base para abordar las dificultades particulares del ambiente MountainCar, donde la recompensa es escasa y la exploración es especialmente difícil.

- **Double DQN:** apunta a corregir la tendencia de DQN estándar a sobreestimar los valores Q. El problema surge porque la misma red se utiliza tanto para elegir la mejor acción siguiente como

para evaluarla, lo que genera aprendizaje inestable. La solución consiste en usar la red online para seleccionar la acción y la red target para evaluarla, reduciendo así la sobreestimación.

- **Actualización suave de la red target:** En DQN base, la red target se actualiza de forma abrupta cada cierta cantidad de pasos, lo que introduce inestabilidad. En cambio, con la actualización suave controlada por un parámetro  $\tau$ , la red target sigue lentamente a la red principal según  $\theta_{target} = \tau \cdot \theta_{online} + (1 - \tau) \cdot \theta_{target}$ , haciendo que el objetivo cambie de manera gradual.
- **Reward shaping:** En MountainCar el agente recibe una penalización de -1 por cada paso y solo obtiene información útil al llegar a la meta, lo que dificulta la exploración y ralentiza el aprendizaje. Para mitigar esto, se agregan recompensas intermedias: se añade un bonus en función de la velocidad ( $r += 3|v|$ ) y por alcanzar el objetivo ( $r += 10$ ).
- **Prefill del replay buffer:** En DQN se comienza a aprender únicamente con experiencias generadas por una política casi aleatoria, lo que reduce la eficiencia de muestras. Para mejorar esto, se llena inicialmente el buffer con episodios exitosos generados por una política heurística: si la velocidad del agente es positiva, empuja hacia la derecha; si es negativa, empuja hacia la izquierda. Además, se incorpora una pequeña exploración mediante un  $\varepsilon$  de prefill para garantizar diversidad en las experiencias iniciales.
- **Retornos de n pasos:** En DQN base, cada transición utiliza la recompensa inmediata y un único paso de bootstrap, lo que propaga la información de forma lenta cuando la recompensa relevante aparece al final del episodio. Con retornos de n pasos, se acumulan n transiciones en un buffer temporal antes de almacenarlas, permitiendo que la información sobre recompensas futuras se propague más rápidamente.
- **Actualización cada 4 pasos:** Optimizar la red en cada paso puede aumentar el costo computacional y favorecer actualizaciones demasiado correlacionadas con la experiencia más reciente. Al ejecutar la optimización solo una vez cada 4 pasos, el entrenamiento resulta más eficiente y estable.
- **Huber loss:** La función de pérdida MSE penaliza fuertemente los errores grandes, lo que puede generar gradientes explosivos cuando los valores Q son imprecisos al inicio del entrenamiento. La Huber loss combina el comportamiento cuadrático para errores pequeños con el lineal para errores grandes, resultando en un entrenamiento más robusto ante outliers en el replay buffer.
- **Gradient clipping:** Incluso con Huber loss, los gradientes pueden crecer de forma descontrolada en ciertos pasos. El gradient clipping limita la norma del gradiente a un valor máximo antes de cada actualización de pesos, evitando que actualizaciones extremas desestabilicen la red.

Con estas mejoras incorporadas, el agente alcanzó los siguientes resultados en el ambiente MountainCar: una recompensa promedio de  $97.25 \pm 8.47$  sobre 100 episodios, de  $99.00 \pm 8.11$  sobre 1000 episodios, y de  $99.17 \pm 8.01$  sobre 10000 episodios.

## 2. Conclusión

Los experimentos sobre los ambientes custom confirmaron que tanto Q-Learning como DQN fueron implementados correctamente: en todos los casos los Q-Values convergieron a los valores esperados y las recompensas se comportaron según lo previsto.

Para FrozenLake, la búsqueda de hiperparámetros permitió alcanzar un 76 % de éxito sobre 100 episodios. La tabla Q converge a la política óptima en prácticamente todas las celdas, con excepción de una que el agente visita con poca frecuencia, lo que dificulta que aprenda la acción correcta para esa posición.

En CartPole obviando ciertas oscilaciones finales, DQN converge en menos episodios que REINFORCE, pero es considerablemente más lento en wall-time y mucho más sensible a los hiperparámetros. Se exploraron mejoras como Huber loss, gradient clipping y un scheduler de learning rate, además de búsqueda automática con Optuna, pero no fue posible eliminar del todo las oscilaciones en la política aprendida. La principal dificultad del trabajo estuvo en lograr que DQN alcanzara una política estable en CartPole, debido a su alta sensibilidad a los hiperparámetros.

Finalmente, las mejoras incorporadas al DQN para MountainCar lograron resultados satisfactorios. La búsqueda de hiperparámetros se realizó de forma manual, por lo que es probable que un proceso más sistemático permitiera mejorar aún más el rendimiento. Para trabajos futuros, se podrían incorporar las mejoras sobre DQN de forma incremental, buscando sus hiperparámetros y observando su resultado. Ese enfoque facilitaría entender la contribución individual de cada componente y obtener configuraciones más robustas. Por último, para el prefill del replay buffer, sería interesante llenarlo con un experto o con una versión entrenada del mismo agente y observar su resultado.